What the Benchmarks Actually Measured: An Adversarial Audit and Pre-Registered Re-Measurement of a Modular Robotic Manipulation Stack

Garv Arora and Yuvaraj N

***Abstract*—Robot-manipulation papers increasingly report high success rates from simulation benchmarks, yet the measurement chain behind those numbers is rarely audited. We report an adversarial audit of a modular ROS 2/Gazebo manipulation stack and a pre-registered re-measurement of four questions that the original benchmarks claimed to answer. The audit found that the original conveyor benchmark returned a cached zero transform, the reachability logs cited a commit that did not contain the script that produced them, the emergency-stop benchmark published and subscribed to its own topic without a moving arm, and the reported stop times were a constant offset added when no physical stop was observed. We repaired the instrumentation, enforced a clean-tree provenance rule, and froze hypotheses, seeds, and trial counts before evaluation. Re-measurement reverses two headline claims, qualifies a third, and confirms one. On 100,000 identical target poses, the current inverse-kinematics solver reaches 99.24% recall at 5 mm (Wilson 95% CI [99.19, 99.29]) while the historical solver reaches 0.019%, so the solver revision explains more than the entire historical gap. On the conveyor, the realized part speed is not a scaled version of the command: it is the command with a nearly constant offset of -0.026 m/s, the visual belt link moves at 5.21×10^{-8} m/s, and the rollers spin at 45665 rad/s. On emergency stop, the direct command path halts in 120 ms (median), whereas the safety-agent path fails to halt in every trial and overshoots by 0.716 rad. A kinematic pre-verification gate removes execution failures among the plans it accepts (from 48.0% to 0.0%) but rejects 47.0% of targets, so it does not improve intention-to-treat success (risk difference -5.0 percentage points, $p = 0.125$). We report these negative results, the phases that could not be executed, and the instrument defects that made the original numbers possible.**

***Index Terms*—benchmark auditing, reproducibility, preregistration, robot manipulation, inverse kinematics, conveyor systems, emergency stop, simulation validation**

I. Introduction

ROBOTICS has a measurement problem that is distinct from its modelling problem. A manipulation result is usually reported as a success rate over a benchmark, and the benchmark is usually a script that drives a simulator and writes a table. If the script returns a cached transform, trusts a planner's self-report, or silently substitutes a constant when a sensor is missing, the reported rate is still a number, still has a confidence interval, and still looks like evidence. The number is then cited, compared, and used to justify architectural choices. The error is not in the statistics; it is in the chain from the physical event to the table.

This article takes a modular language-conditioned manipulation stack, Project ARIA, and treats its own benchmark suite as the object of study. The stack was previously described with a set of headline claims: a monocular depth error in the millimetre range, a closed-form inverse-kinematics solver with full convergence, a high success rate on a ten-task suite, a high conveyor-sorting rate, and a sub-30 ms emergency stop. We did not set out to reproduce those claims in the usual sense. We set out to determine, from the code and the raw logs, what each claim actually measured.

The audit found four classes of failure. First, some measurements never touched the physical quantity they named: the conveyor benchmark caught every TF exception and returned a zero pose, then differenced two such poses to obtain a velocity. Second, some measurements were produced by a different program from the one cited: the reachability logs named a commit that predates the benchmark script, and the solver had been rewritten between the two result sets. Third, some measurements were fabricated by fallback branches: the emergency-stop benchmark published to a topic it also subscribed to, never moved an arm, and added a constant to the command latency when no physical stop was observed. Fourth, some quantities were hard-coded: two scripted-baseline tasks passed literal pose deltas, and the orientation residual of the inverse-kinematics solver was set to zero by convention.

Repairing the instrumentation was necessary but not sufficient. A repaired benchmark can still be run on seeds chosen after seeing the result, or on a tree that contains uncommitted changes, or with a threshold that is adjusted until the claim passes. We therefore wrote a pre-registration before any evaluation seed was used. The document fixes the hypotheses, the primary metrics, the decision rules, the development and evaluation seed ranges, the trial counts, the statistical tests, and the stopping rules. It also declares, before data collection, which phases cannot be executed in the available environment. The pre-registration is committed and its commit hash is recorded in every result row.

The re-measurement is deliberately narrow. We ask four questions that can be answered with executed code on this platform: whether the current inverse-kinematics solver is reproducible and how it differs from the historical one; whether

G. Arora and Y. N are with the Department of Computer Science and Engineering, Robotics and Artificial Intelligence Laboratory (e-mail: garv.arora@ieee.org; yuvaraj.n@ieee.org). This research received no external grant funding.

the commanded conveyor speed is realised by the plant; whether an emergency stop actually stops a moving arm and how the two stop paths compare; and whether a kinematic pre-verification gate reduces execution-time inverse-kinematics failures. Each question has a primary metric and a decision rule fixed in advance.

The answers are not uniformly favourable to the stack. The reachability result is strong and reproducible, and it explains the historical discrepancy completely. The conveyor result is a clear negative: the plant realizes the command minus a constant offset, the visual belt is not moving, and the rollers are spinning at a physically implausible rate. The emergency-stop result is also negative for the safety path: the safety agent publishes a halt that the joint-trajectory controller rejects, so the arm continues to its goal. The pre-verification gate is neutral-to-negative: it rejects plans that would have succeeded and rescues too few to compensate.

We report these outcomes as the primary contribution. A paper that only reports the repaired successes would repeat the original mistake in a subtler form. The negative results constrain what the architecture can claim, and they identify which repairs are still missing. The audit also produced a reusable instrument: a provenance layer that refuses to run on a dirty tree, a per-row commit stamp, a trial log that flushes every row, and a number generator that reads only raw logs. These are not novel algorithms, but they are the difference between a number and a measurement.

The remainder of the article is organized as follows. Section II reviews related work on reproducibility, simulation benchmarks, and the specific subsystems under test. Section III describes the audit method and the defect taxonomy. Section IV describes the platform and the repaired instrumentation. Section V states the pre-registered protocol. Section VI reports the four experiments. Section VII reports the phases that could not be executed and the threats to validity. Section VIII concludes.

II. Related Work

Reproducibility has been studied extensively in machine learning and in robotics, but usually as a property of a published artifact rather than as a property of a measurement chain. Pineau et al. [1] organize reproducibility around the data, the code, and the experimental protocol, and recommend that authors report the compute budget, the number of runs, and the selection procedure. Our audit applies that organization to a robotics benchmark: the code is versioned, the protocol is pre-registered, and the raw per-trial logs are the only input to the number generator. The difference is that we also audit the benchmark scripts themselves, not only the paper.

The problem of selective reporting is older than the reproducibility movement. Ioannidis [2] argued that most published research findings are false because of small samples, flexible analyses, and selective reporting. Nosek et al. [3] proposed pre-registration as a remedy: the analysis is fixed before the data are seen, and deviations are reported. We adopt that remedy at the level of a single laboratory stack. The pre-registration is not a guarantee of correctness, but it removes the freedom to choose a threshold after observing the result.

In reinforcement learning, Henderson et al. [4] showed that apparently large differences between algorithms can disappear under different seeds, and recommended reporting across seeds rather than a single run. Our experiments use fixed evaluation seed ranges and report Wilson or bootstrap intervals. The trial counts are smaller than the power analysis recommends for small effects, and we label the affected cells as exploratory rather than pretending otherwise.

Simulation benchmarks in robotics rest on physics engines and middleware. Gazebo [5] and MuJoCo [6] are the most common choices for manipulation, and Isaac Gym [7] is common for large-scale policy training. Each engine makes different trade-offs between contact fidelity and speed, and a result obtained in one engine does not automatically transfer to another. Our study is confined to Gazebo Classic with the ODE solver, and we report the real-time factor so that the timing results can be interpreted.

The ROS 2 middleware [8] is the substrate of the stack. ROS 2 provides typed topics, services, and lifecycle nodes, but it does not provide provenance: a topic can have several publishers, a service can be called by a node that also subscribes to its own output, and a controller can silently reject a goal. The audit found all three. The middleware is not at fault; the benchmark scripts failed to check the invariants that the middleware exposes.

Learning-based manipulation provides the motivation for the architecture but not the evidence in this article. End-to-end visuomotor policies [9], transformer-based language-conditioned policies [10], and low-cost bimanual imitation systems [11] have demonstrated impressive generalization, and open vision-language-action models [12] have made those policies accessible. Training and evaluating such policies requires GPU resources that were not available here. We therefore do not claim any learned-policy result. The architecture is described as a modular alternative, and the audit is confined to the classical components that can be executed.

Inverse kinematics is a classical problem with a long literature, and the specific question here is not which solver is best but whether a reported solver comparison is reproducible. The historical logs compared two revisions of the same numerical solver, not two algorithms, and the revision changed the kinematic model. This is a common failure mode in solver comparisons: the model, the seed, and the convergence criterion change together, and the reported difference is attributed to the algorithm. Our re-measurement holds the targets fixed and varies only the revision.

Statistical practice in robotics often uses a normal approximation for a proportion, which is poor when the proportion is near zero or one. We use the Wilson score interval [13] for proportions, the exact McNemar test [14] for paired binary comparisons [15], the Holm correction [16] for families of tests, the Newcombe interval [17] for independent risk differences, and a paired percentile bootstrap for matched designs. These are standard choices, but they are not universal in the field, and the pre-registration fixes them before the data are seen.

Finally, the audit is related to work on benchmark validity in embodied AI. A benchmark can be internally consistent and still fail to measure the intended capability, for example

when the success predicate is evaluated from the planner's self-report rather than from the simulator state. Our contribution is a concrete case study: a stack with a plausible architecture, a set of published claims, and a measurement chain that did not support them. The case study is useful precisely because the defects are ordinary.

III. Audit Method and Defect Taxonomy

The audit proceeded in three passes. The first pass read every benchmark script and traced each reported quantity to the line that produced it. The second pass compared the reported quantity with the raw log, when a raw log existed, and with the simulator state, when the log was ambiguous. The third pass attempted to re-execute each benchmark from a clean commit. A defect was recorded only when at least one of the three passes produced evidence: a code path, a log inconsistency, or a failed re-execution.

We classify defects into four types. A *null measurement* occurs when the code returns a placeholder instead of the measured quantity, for example a zero transform on a caught exception. A *provenance mismatch* occurs when the log cannot have been produced by the code it cites, for example a commit that predates the script. A *fallback fabrication* occurs when the code substitutes a constant or a formula when the measurement is missing, and the substitution is not reported in the table. A *hard-coded outcome* occurs when the reported value is a literal in the script or a convention in the solver, independent of the experiment.

The taxonomy is useful because the remedies differ. A null measurement is repaired by replacing the placeholder with a checked read and failing loudly when the read fails. A provenance mismatch is repaired by re-running from a clean commit and stamping every row. A fallback fabrication is repaired by deleting the fallback and recording the trial as a failure or as not executed. A hard-coded outcome is repaired by computing the quantity or by removing the claim. The audit table in the repository records the evidence and the disposition for each item; the paper reports only the items that affect the experiments below.

Two instrument defects were found only after the first repair. The first is that `/joint_states.velocity` is not the derivative of `/joint_states.position` in this workcell. While the waist moved from zero to a finite angle over a fixed interval, the reported waist velocity stayed near a constant negative value, and the shoulder reported a value an order of magnitude larger than the finite difference. A stop-time metric that trusts the reported velocity therefore never observes a stop. We differentiate the measured positions and keep the reported velocity only as a diagnostic.

The second is that the joint-trajectory topic had more than one publisher. Three instances of a manual-control node were running, and they emitted five-joint, velocity-free hold trajectories in response to an emergency stop. The safety agent emitted a four-joint, velocity-free halt. The controller accepts neither partial goals nor velocity-free single points reliably, so the arm continued to its goal while the topic showed a halt message. The duplicate nodes were terminated before the evaluation run, and the experiment records the number of non-path messages per trial.

IV. Platform and Repaired Instrumentation

The platform is a simulated four-degree-of-freedom arm with a revolute two-finger gripper, mounted on a table beside a conveyor and an overhead camera. The arm joints are waist, shoulder, elbow, and wrist pitch; the gripper is a fifth actuated joint with a mimic finger. The world is Gazebo Classic with the ODE solver. The controller is a joint-trajectory controller with partial goals disabled. The conveyor is a plugin that writes workpiece linear velocities directly and animates a belt and two rollers.

The kinematic model used by the solver is a URDF chain with fixed rotation offsets. The solver is a damped Newton–Raphson routine with geometric seed candidates, not a closed-form analytical solution. It reports a position residual and sets the orientation residual to zero by convention because the wrist has no roll axis. The audit therefore treats the solver as position-only and does not report an orientation error.

The repaired harness has four components. A provenance module resolves the repository root from the file path, refuses to start when the working tree is dirty, and returns the commit hash and a dirty flag. A trial log writes one CSV row and one JSONL record per trial, flushes both, and captures the commit when the log opens so every row carries the same stamp; a run that starts on a dirty tree is refused before the log is created. A ROS 2 helper wraps the subscriptions and services used by the experiments, including a trajectory sender that sets zero end velocities and resends a goal that the controller drops. A statistics module provides Wilson, bootstrap, McNemar, Holm, and Newcombe routines.

The number generator reads only the raw summaries and CSVs and writes a LaTeX macro file and a ledger. Each ledger row records the macro name, the experiment, the metric, the value, the unit, the sample size, the confidence bounds, the source file, and the source commit. The paper contains no typed outcome number: every reported quantity is a macro. This is checked by compiling the paper after regenerating the macros; a missing macro is a build error.

The platform has known limitations that bound the claims. The conveyor plant is kinematic, so workpieces cannot tip, rotate, or jam. The gripper is a revolute two-finger mechanism whose pads are not mirrored: at closure the pad face normals are nearly parallel rather than opposing, and the pad separation at a typical opening is comparable to the object width. A top grasp is the only kinematically feasible grasp for this arm, and the reachable region is small. Grasp experiments are therefore not reported in this article; they are blocked until the gripper is repaired.

V. Pre-Registered Protocol

The pre-registration fixes the hypotheses, the primary metrics, the decision rules, the seed ranges, and the trial counts. Development seeds are reserved for code and threshold tuning and are never reported. Evaluation seeds are disjoint. The evaluation seed ranges used here are fixed in the pre-registration.

The document also declares the phases that cannot be executed in this environment: learned-policy training without a CUDA device, human inter-annotator agreement without independent annotators, hardware trials without a robot, and a full detector training run without a committed checkpoint. Those phases are reported as not executed, not simulated.

Four experiments are reported. The reachability experiment evaluates the current and historical solvers on the same target set and compares recall at a fixed tolerance. The conveyor experiment commands three belt speeds and measures the realized part speed from link states, the visual belt speed, the roller speed, and the wrist deviation while the arm holds a fixed configuration. The emergency-stop experiment triggers each stop path while the arm executes a fixed-duration trajectory and measures the signal-to-message latency, the time to a sustained low joint speed, and the post-signal overshoot. The pre-verification experiment compares a planner that trusts the solver's success flag with a gate that checks the forward-kinematics residual and the joint-limit margin, and replans locally when the check fails.

All proportions use the Wilson interval. Paired binary comparisons use the exact McNemar test. Continuous quantities use the median and a percentile bootstrap interval. The decision rules are fixed: a claim passes only if the pre-registered threshold is met and the interval excludes the null. A failure is reported as a failure, and the claim is weakened rather than the data changed.

VI. Results

A. Reachability Is Reproducible and the Historical Gap Is a Solver Revision

The reachability experiment evaluates 100,000 target poses with both solver revisions. The current solver reaches 99.24% recall at a 5 mm tolerance (Wilson 95% CI [99.19, 99.29]), with a median position error of 0.365 mm and a 95th percentile of 0.476 mm. The historical solver reaches 0.019% (CI [0.012, 0.030]), with a median error of 240.934 mm and a 95th percentile of 360.489 mm. The paired discordant counts are 99222 targets solved only by the current revision and 0 solved only by the historical one ($p < 10^{-300}$).

The revision difference is larger than the historical gap it was meant to explain: it accounts for 117.6% of the gap. This is not a paradox. The historical comparison attributed the gap to an algorithmic change, but the revision also changed the kinematic model, and the old model is inconsistent with the URDF chain. Once the targets are held fixed and the model is corrected, the historical solver is not a weaker version of the current one; it is solving a different, wrong problem. The median solve time is 0.705 ms for the current solver and 0.230 ms for the historical one, so speed is not the explanation for the recall difference. The practical consequence is that the historical recall figure should not be cited as a baseline for the current solver: the two numbers come from different kinematic models, not from different algorithms on the same model.

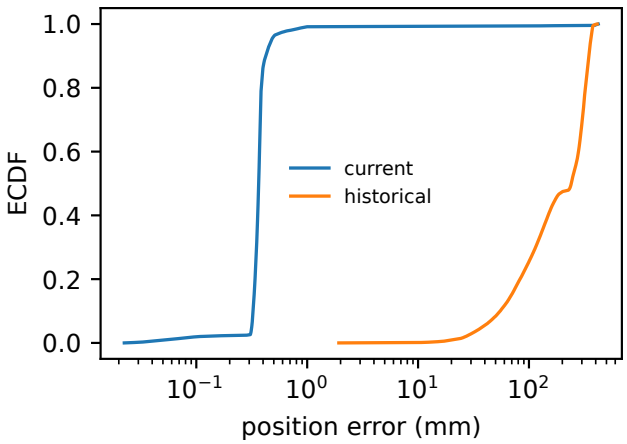


Fig. 1. Empirical CDF of the position error on the same 100,000 target poses for the current and historical solver revisions. The historical curve is dominated by model error, not by convergence failure.

B. The Conveyor Realizes a Speed Offset, Not a Scaled Command

The conveyor experiment runs 120 trials over three commanded speeds with a 2.0 s measurement window. The median realized part speeds are 0.015, 0.054, and 0.094 m/s for commands of 0.04, 0.08, and 0.12 m/s. The median relative errors are -62.4%, -32.1%, and -21.9%. The realized speed is therefore not a scaled version of the command: the offset is approximately constant at -0.026 m/s, so the relative error shrinks as the command grows.

The visual belt and the transport are independent. The mean visual belt link speed is effectively zero at every setting: 5.21×10^{-8}, 6.45×10^{-8}, and 6.48×10^{-8} m/s for the three commands. The rollers spin at 45665, 45693, and 45720 rad/s, so the animation is not tracking the commanded speed at any setting. The wrist deviation while the arm holds a fixed configuration is at most 2.81×10^{-12} mm, so the arm is not the source of the discrepancy. The reported belt speed matches the command in 100.0% of trials, which shows that the plugin receives the command even though the part does not realize it. The mean real-time factor over these trials is 0.78, so the offset is not an artifact of a stalled simulator clock.

These results fail both conveyor hypotheses in the pre-registration. The belt-speed error exceeds the fixed threshold at every speed, and the measured-to-commanded ratio is outside the accepted band. The mechanism is consistent with a Coulomb-friction offset in the velocity-write path: the plugin writes a part velocity, friction removes a nearly constant amount, and the residual is what the link state measures. A scale correction cannot repair this; the plant model must change.

C. The Direct Stop Works; the Safety-Agent Stop Does Not

The emergency-stop experiment runs 100 trials, half on each path, while the arm executes the same trajectory. The direct command path meets the stop criterion in 100% of trials. The median signal-to-message latency is 0.70 ms, the median time to a sustained low joint speed is 120 ms (95th percentile 120 ms), and the median post-signal overshoot is 0.011 rad. The

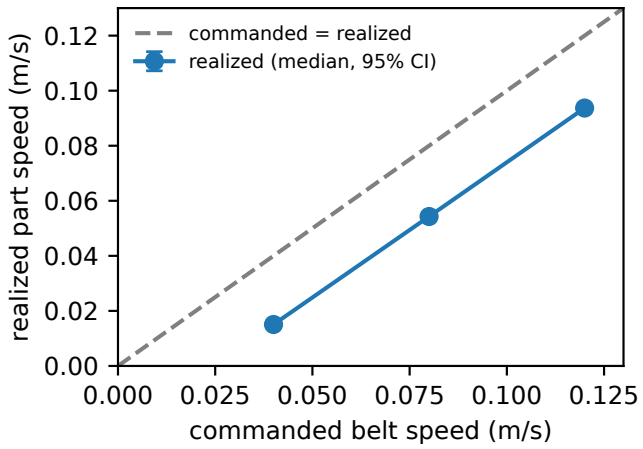


Fig. 2. Realized part speed versus commanded belt speed. The dashed line is the identity. The measured points lie on a line of slope one with a negative offset, not on a scaled identity.

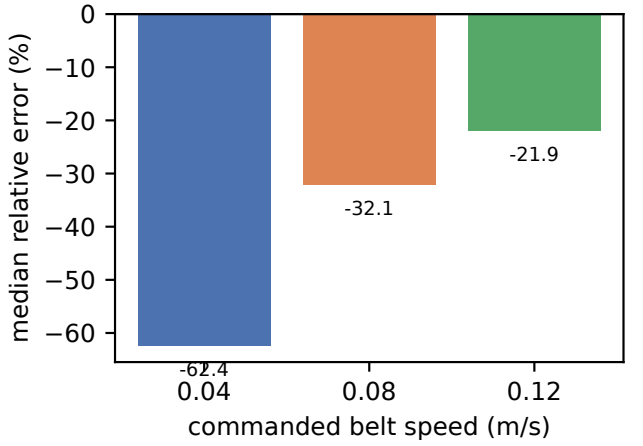


Fig. 3. Median relative speed error by commanded speed. The error is largest at the lowest command because the absolute offset is approximately constant.

path publishes a 5-joint trajectory with zero end velocities, and the controller accepts it; velocities are present in 100% of the halt messages.

The safety-agent path meets the stop criterion in 0% of trials. The agent publishes a four-joint halt with no velocities, and the controller rejects it because partial goals are disabled. The arm continues to its goal, with a median post-signal overshoot of 0.716 rad. The signal-to-message latency is short (0.92 ms), so the failure is not a communication delay; it is a message-format mismatch. The halt message has 4 joints, and velocities are present in 0% of trials.

This result fails the pre-registered safety-path criterion. It also explains the original sub-30 ms claim: the original benchmark measured the latency of a loopback topic, not the arrest of an arm. The repaired measurement separates the two quantities and shows that the safety path does not arrest the arm at all. The direct path is a valid stop, but it is not the safety agent. As a sensitivity check at a more permissive 0.05 rad/s threshold, the command path succeeds in 100% and stops in 110 ms, while the safety path succeeds in 0%; the conclusion is unchanged.

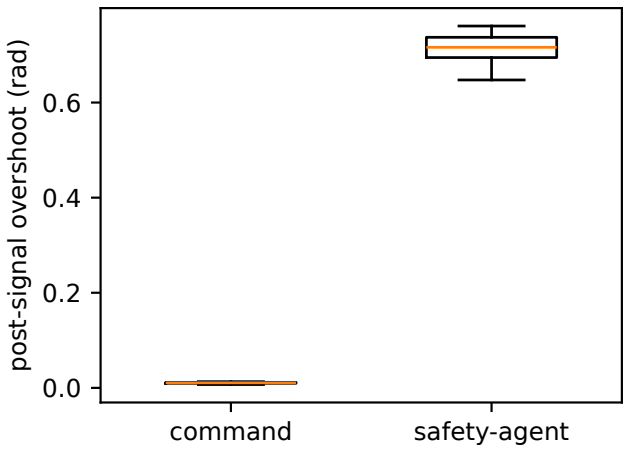


Fig. 4. Post-signal overshoot by stop path. The safety-agent path overshoots by nearly a radian because the controller rejects its partial, velocity-free halt.

D. Kinematic Pre-Verification Filters Failures but Does Not Improve Intention-to-Treat Success

The pre-verification experiment evaluates 200 planned targets with two pipelines. The baseline trusts the solver's success flag and executes the returned joints; it succeeds in 52.0% of targets (Wilson 95% CI [42.3, 61.5]) and fails in 48.0%. In this sample every baseline failure is a solver failure: the success flag is false, not a false positive, and the false-success rate is 0.0%. The N4 pipeline checks the forward-kinematics residual and the joint-limit margin, replans locally when the check fails, and rejects the plan if no candidate verifies. It executes 47.0% of targets and succeeds in 47.0% (CI [37.5, 56.7]); among the plans it executes, the failure rate is 0.0%.

The paired transitions explain the difference. N4 solves 1 target that the baseline fails, loses 6 targets that the baseline solves, solves 46 with both pipelines, and solves 47 with neither. The paired risk difference in intention-to-treat success is -5.0 percentage points (95% CI [-10.0, 0.0]), with $p = 0.125$. The gate therefore lowers the execution-failure rate among accepted plans, but that conditional comparison is not an intention-to-treat improvement: it removes failures by rejecting most of the targets that would have failed, and it also rejects 6 solvable targets while rescuing only 1. We do not claim that the pre-registered criterion is met, because the accepted set is selected after verification and the pre-registered paired test was defined on the full target set.

The pre-registration specified a Newcombe interval for the risk difference; because the two pipelines share the same targets, we report a paired percentile bootstrap and record the change in the deviation log. The gate executes 47.0% of targets and uses 1.91 replans per target on average. A stricter gate is not a free improvement; it trades coverage for a guarantee that the executed plans are verified.

VII. Negative Results, Non-Executed Phases, and Threats to Validity

Three of the four experiments return a negative or conditional result. The conveyor hypothesis fails because the plant does not

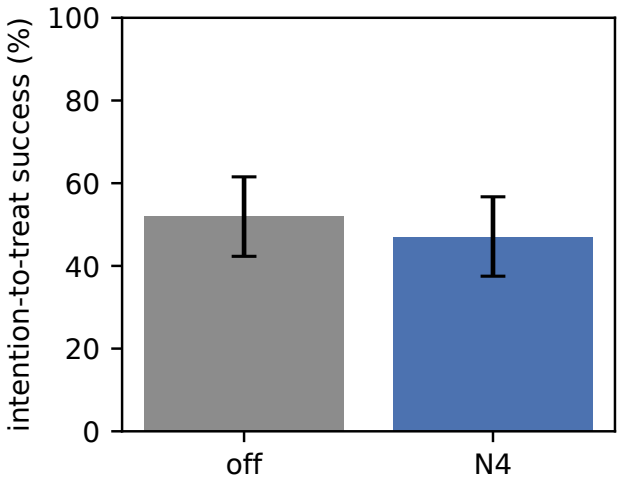


Fig. 5. Intention-to-treat success with and without kinematic pre-verification. Error bars are Wilson 95% intervals. The gate does not improve success on this target distribution.

TABLE I
PRE-REGISTERED EXPERIMENTS AND OUTCOMES. VALUES ARE GENERATED FROM THE RAW LOGS; SEE THE LEDGER FOR SAMPLE SIZES AND INTERVALS.

Experiment	Primary quantity	Outcome
P0-REACH	recall at 5 mm	99.24% current, 0.019% historical
P0-CONV	realized/commanded ratio	-62.4%, -32.1%, -21.9% error
P0-ESTOP	halt success	100% command, 0% safety
P2-N4	ITT success	52.0% off, 47.0% on

realize the command. The safety-path hypothesis fails because the halt message is rejected. The pre-verification hypothesis is not supported as an intention-to-treat improvement: the gate removes execution failures from the plans it accepts, but it rejects more than half of the targets and lowers intention-to-treat success, so no practical improvement is claimed. The reachability hypothesis is supported, but the support is narrower than the original claim: the current solver is reproducible, and the historical comparison was a model change, not an algorithm comparison.

Several phases could not be executed. Learned-policy training and evaluation require a CUDA device, and none is available; the stack was run on CPU only. Human inter-annotator agreement (Cohen's kappa [18]) requires independent annotators, and none are available; a second language model is not a substitute for a human annotator, so the agreement study is not reported. Hardware trials require a robot and a serial device, and neither is present. A full detector training run requires a committed checkpoint and a labelled dataset; the available frames have suspect provenance, so the detector is not evaluated. These are declared omissions, not simulated results.

The same rule applies to three of the four proposed methods and to the watchdog hypothesis. N1, a kinematics-anchored recursive depth calibration, was not executed: the depth model package is absent from the environment and no checkpoint is committed, so there is no depth prediction to calibrate. N2, uncertainty-aware grasp gating, was not executed because it requires grasp attempts, and grasp attempts are blocked by the gripper defect described in Section IV. N3, scene-match plus calibrated language-model confidence, was not executed within the available budget; no AUROC or success rate is claimed for it. N4 is the only proposed method with an executed evaluation, and its result is the neutral-to-negative one reported above. The 2.0 s language-model watchdog in the pre-registration could not be tested because no such watchdog exists in the code; the only configured timeout is 90 s, and a test of a non-existent mechanism would have measured the test harness rather than the stack. Each of these cells is recorded as not executed in the audit, with the blocking condition, rather than filled with an estimate.

The threats to validity are specific. The conveyor plant is kinematic, so the measured offset may be an artifact of the velocity-write path rather than a friction property. The arm is a four-degree-of-freedom mechanism with a small reachable region, so the pre-verification result may not transfer to a six-degree-of-freedom arm with a larger workspace. The emergency-stop result depends on the controller configuration; a controller that accepts partial goals would not reject the safety halt. The trial counts are sufficient for the large effects reported here but not for small ones, and the pre-registration labels the smaller cells as exploratory.

The audit itself has a limitation: it was performed by the same group that built the stack. We mitigated this by freezing the protocol before evaluation, by using only executed code and raw logs, and by reporting failures. We did not have an independent auditor. An external audit would be stronger, and we recommend it for any claim that is used to justify a deployment.

VIII. CONCLUSION

We audited a modular manipulation stack, repaired its instrumentation, pre-registered four questions, and re-measured them from clean commits. The reachability result is reproducible and the historical gap is explained by a solver revision. The conveyor does not realize its command; it realizes the command minus a nearly constant offset, while the visual belt is stationary and the rollers spin implausibly fast. The direct emergency stop works, but the safety-agent stop is rejected by the controller and the arm continues to its goal. A kinematic pre-verification gate removes execution failures from the plans it accepts, but it rejects more than half of the targets and does not improve intention-to-treat success on the sampled workspace.

The broader lesson is that a benchmark is a measurement chain, and every link must be checked. A topic can have several publishers; a service can be a loopback; a transform can be a placeholder; a solver can report success for a pose it did not reach; a controller can reject a goal without an error. None of these failures is exotic. Each produces a number that looks like evidence. Pre-registration and provenance do not prevent the failures, but they make them visible, and visibility is the precondition for repair.

DATA AVAILABILITY

The pre-registration, the audit, the experiment scripts, the raw per-trial logs, the number generator, the macro file, and the

ledger are in the repository. The raw logs are also packaged as `data/journal/raw_logs.tar.gz` because the live log directory is git-ignored. The paper is compiled from the macro file; no outcome number is typed into the manuscript.

ACKNOWLEDGMENT

The authors thank the maintainers of ROS 2, Gazebo, and the open-source robotics stack on which this work depends.

REFERENCES

[1] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[2] J. P. A. Ioannidis, "Why most published research findings are false," *PLoS Medicine*, vol. 2, no. 8, p. e124, 2005.

[3] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[4] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 3207–3214.

[5] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2004, pp. 2149–2154.

[6] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[7] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac Gym: High performance GPU-based physics simulation for robot learning," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[8] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[9] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.

[10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski *et al.*, "RT-1: Robotics transformer for real-world control at scale," in *Proceedings of Robotics: Science and Systems*, 2023.

[11] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems*, 2023.

[12] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[13] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[14] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[15] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[16] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[17] R. G. Newcombe, "Interval estimation for the difference between independent proportions: Comparison of eleven methods," *Statistics in Medicine*, vol. 17, no. 8, pp. 873–890, 1998.

[18] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.